

ccBPF in LTTit: From Dynamic Programming to Execution Migration and Scheduling Research

Skai Uijing
LTTit Research

Research record, October 6, 2026

Abstract—LTTit investigates how small heterogeneous devices can expose resources and execute programs as a shared distributed runtime. ccBPF began as a C-subset compiler and a BPF-derived extension mechanism; explicit interpreter state subsequently enabled a running invocation to suspend and continue on another node. This paper reconstructs that progression from design posts, source code, board demonstrations, and archived host experiments. We distinguish dynamic code installation, migration of an active execution, and the still-open decision of where execution should run. The current configured context occupies 528 bytes; its meaning depends on the program image, Native bindings, and external resources. A conditional continuation argument explains what the saved state preserves, while source inspection identifies what it does not. The October 4 corpus records 7,518 checkpoint restorations and ten fresh-process handoffs, without establishing general compiler correctness or MCU migration latency. We also retain the exploration of AIMD-based distributed resource control and identify an algebraic gap in the local scheduling notes. No automatic placement algorithm or global scheduling guarantee is claimed. The result is an evidence-based account of an implemented mechanism and an unfinished research direction.

Index Terms—embedded runtime, bytecode, execution migration, distributed operating systems, resource control

I. RESEARCH MOTIVATION AND DEVELOPMENT

LTTit’s early design notes ask whether many small devices can be used as one programmable system. The March 11 outline separates a replaceable local RTOS from distributed communication, a resource view, and dynamic programming. The March 29 account further distinguishes CCNET routing, SCP reliable transport, and CCRPC file-like remote operations. These are architectural intentions alongside working components, not evidence that membership, consistency, transparent migration, and fault recovery were all implemented.

The initial ccBPF objective was practical: change deployed behavior without rebuilding and reflashing the entire host. A small source compiler and interpreter allow extension logic at predefined hook points. March’s usage and compiler posts describe compiling at a resource-rich node and loading an image elsewhere, with optional compilation on a small device. The June 5 account connects this to distributed execution: if program state is explicit in a VM, it can be packed and resumed elsewhere. It also records debugging and instrumentation as motivations, rather than presenting migration as the project’s only original purpose.

The resource model is equally important. LTTit names remote resources through a world-tree namespace and file-style

TABLE I
RESEARCH PROGRESSION AND THE EVIDENCE SUPPORTING IT.

Stage	Evidence and scope
March design	Modular runtime, CSC stack, remote operations, C-subset compiler and VM; several cluster services remain plans.
June exploration	Hooks, instrumentation, shared resource names, and explicit VM-state movement. Early safety language is a design aspiration, not a proof.
Board integration	Pico 2W sends image/context through CCRPC; STM32 resumes a demonstration program.
August notes	Distributed computation viewed as rate allocation; AIMD literature explored, without a completed migration policy.
October 4 corpus	Host checkpoint, fresh-process, representation-size, and primitive-timing records.
Current status	Execution substrate exists; placement, ownership recovery, and resource-transfer semantics remain open.

operations. This allows a program to refer to a remote resource without embedding its physical host into every operation. It does not imply that moving a VM automatically moves an open file, a device session, or a map. Nor is the world tree the physical communication tree optimized by CCNET. Keeping those two meanings of tree separate is necessary to understand what migration actually changes.

This paper records that history and audits its evidence. It does not substitute a newly invented task-placement objective for the author’s unfinished scheduling research. The inspected LTTit revision is b0941d4; the current ccBPF repository head is 4432ea4, while the archived host experiment identifies 18aad430. Working-tree manifests retain selected source hashes, because a commit identifier alone does not describe uncommitted documentation or experiment files.

II. RELATION TO EARLIER SYSTEMS

The BSD Packet Filter introduced a compact packet-processing virtual machine [1]. ccBPF uses a classic-BPF-derived accumulator/index organization and adds its own language, image, and host-call conventions. It is inspired by eBPF’s dynamic-extension use, but does not execute Linux eBPF binaries or inherit Linux’s verifier guarantees.

Embedded reprogramming and migration are established research directions. Maté used compact code capsules for

sensor networks [2]; Agilla provided mobile agents in that setting [3]. Code mobility literature distinguishes moving code from moving an execution and its state [4]. More recent rBPF and Femto-Containers study BPF execution and isolation on microcontrollers [5], [6]; WARDuino explores dynamic WebAssembly execution on microcontrollers [7]. These precedents prevent claiming that an embedded VM, dynamic loading, or execution migration is new by itself.

The project’s specific engineering line is a dedicated C-subset compiler connected to a small interpreter, named host services, hook attachment, and a transferable continuation. Its evidence is functional and empirical. Mechanized interpreter work such as CertrBPF [8] sets a different assurance level; passing the ccBPF corpus does not establish refinement, isolation, or safety for arbitrary untrusted inputs.

The scheduling investigation also has direct predecessors. Vlahakis, Athanasopoulos, and McLoone study AIMD scheduling and resource allocation in distributed computing [9]. Thus reusing network feedback ideas for computational resources is a research lineage to explain, not an originality claim. The open LTTit question is how such control should interact with migratable executions, remote resource access, and the cost of moving state.

III. COMPILER AND RUNTIME CONTRACT

A. From Source to a Loadable Image

The compiler accepts a restricted C language, builds front-end representations, lowers through intermediate code, and emits BPF-derived instructions plus program data. The loader reconstructs an in-memory program with instruction and data pointers, an entry point, and string information. Those host addresses are loader products; they are not a portable continuation format.

The compiler and VM share a memory-layout contract. Local values, temporaries, arguments, and host-call results are placed in interpreter-managed storage according to that contract. This makes the scope of the saved execution state inspectable. It also limits expressiveness: unrestricted native stacks, arbitrary host pointers, and external device state cannot simply be copied and expected to work elsewhere.

A *Native* is a registered host function accessible to bytecode through a numeric identifier. The destination resolves the identifier to its own function implementation. Equal identifiers are necessary but not sufficient for portability: their argument conventions, side effects, and results must match the source program’s expectations. A function that opens a local sensor cannot acquire transparent semantics merely because its identifier is unchanged.

B. Three Different Operations

Loading makes a program image available. *Hook attachment* associates it with a host-defined invocation point; an invocation can start with a fresh context. *Execution migration* instead moves an already started invocation’s context and resumes it. Installing the same image on another node and starting it from the entry point is code deployment, not continuation.

The stepped VM interface accepts a context and an instruction budget, returning statuses such as finished, error, or migration requested. The LTTit demonstration calls it with a budget of 64 instructions. That budget bounds bytecode steps per call; it does not bound elapsed time if a Native blocks or performs variable work. Cooperative stepping is therefore not a hard real-time scheduling proof.

IV. WHAT IS PRESERVED BY A MIGRATION

A. Explicit State and the Handoff Point

At the inspected configuration the context consists of accumulator A , index X , instruction position pc , return value ret , and $B = 512$ bytes of VM working storage. With four 32-bit fields,

$$|S| = 16 + B = 528 \text{ bytes.} \quad (1)$$

This figure describes the context representation, not the total runtime footprint or the program image. Build-time configuration and layout must agree across the two runtimes.

The migration Native completes before the interpreter returns the migration status. The VM saves A , X , and $pc + 1$, so the destination resumes at the next instruction. This detail preserves the already returned Native value for a following store. In the recorded host demonstration, the migration call occupies instruction 86, the next store is 87, and the relevant working-memory location is 144. Restarting the call or skipping the store changes semantics.

Current context packing allocates a buffer and copies the context bytes; unpacking checks the context length and copies them back. It is a same-representation mechanism. It does not establish a versioned endian-neutral wire ABI, check the program identity, or validate all resumed-state invariants. The program image must also be available at the receiver, where the loader reconstructs host pointers and local Native bindings.

B. Conditional Continuation Argument

Let P denote the loaded program semantics, S its explicit VM state, and E the relevant host environment: Native behavior, input bytes, maps, and resource responses. Write one execution step as

$$(S', o) = \delta(P, S, E), \quad (2)$$

where o is the observable effect. This notation states a contract for reasoning about the implementation; it is not a new migration algorithm.

Proposition 1. *Suppose packing and unpacking preserve S exactly, the destination loads the same program semantics, and corresponding future steps have equivalent external inputs and Native effects. Then a deterministic execution resumed at the saved instruction has the same subsequent state sequence and observations as continuing there locally.*

Proof. Immediately after restoration, the instruction position, registers, and working storage are equal. Under equivalent P and E , applying the same transition δ produces equal next states and observations. Repeating this argument inductively

Algorithm 1: Existing cooperative migration path

1. Source loads image and initializes explicit VM state.
 2. Execute bounded instruction steps until migration status.
 3. Suspend source invocation at the saved next instruction.
 4. Pack image length, context length, image, and context.
 5. Write the package to the destination’s migration resource.
 6. Destination loads image, restores context, and continues.
- Missing protocol:* durable ownership transfer, deduplication, compatibility negotiation, and failure recovery.

Fig. 1. Source-derived mechanism pseudocode, not an automatic placement policy. The current example names a fixed destination.

gives equality for every finite common suffix, including termination if reached. The migration Native is not repeated because the saved position follows it. $\square$

The premises delimit the result. External input buffers and map contents are not generally in S . Device handles, files, network sessions, and previous non-idempotent effects belong to the environment. Neither byte equality nor the induction proves the required environment equivalence. The current implementation therefore supports compatible cooperative continuation, not transparent migration of arbitrary native processes or all OS resources.

C. Actual LTTit Path

The Pico-side shell loads an image and executes with an explicit context. On migration status it signals a sender task and stops that execution loop. The sender packs two 32-bit lengths, the image, and the context, then issues a CCRPC write to `/root/nodeB/vm/migrate`. The STM32 handler checks basic lengths, copies the image/context, loads the program, restores the context, and steps the VM.

The inspected handler performs limited framing checks; it is not a complete parser validation argument. The host nodeC/nodeD demonstration uses Unix sockets and native-size framing, including single-call reads/writes that do not constitute a general partial-I/O protocol. These demonstrations establish execution paths under their tested conditions, not a production transfer service.

The board demo is separate from the host fixture. Its source-side sequence precedes destination outputs 5, 6, and a final result 1129. The host demo prints a different sequence and returns 0. Combining the two into one alleged trace would erase their different programs and integration environments.

V. ARCHIVED EVALUATION

A. Questions and Method

The October 4 host corpus asks whether compiled test programs preserve their observable suffix across checkpoints, how large the representations are, and what individual runtime operations cost. It does not evaluate an online scheduler or compare migration decisions against staying local.

The corpus contains 32 source workloads compiled under `-Os` and `-O2`, giving 64 runs. Arithmetic, branches, strings, and cooperative handoff are exercised. Checkpoint tests compare restored continuations with reference executions. Ten

TABLE II

OCTOBER 4 ARCHIVED HOST CHECKS. COUNTS ARE OUTCOMES OF A FINITE CORPUS, NOT PROOFS OF ARBITRARY-PROGRAM CORRECTNESS.

Check	Recorded outcome
Workloads / optimization configurations	32 / 2
Complete workload runs	64/64 successful
Checkpoint restorations	7,518/7,518 matched
Fresh-process handoffs	10/10 matched
Expected input-validation outcomes	448/448
Fixture allocation-balance checks	64/64

TABLE III

ARCHIVED REPRESENTATION SIZES IN BYTES. LOADED CHARGE EXCLUDES VM STATE; IMAGE PLUS STATE IS THE UNFRAMED TRANSFER SIZE. NEITHER IS PEAK RAM. THE BOARD-PROGRAM ROW IS ITS REPRESENTATION IN THE HOST CORPUS.

Program	Insns.	Image	Loaded	Image+S
Host demo	200	1695	1856	2223
Board demo program	112	928	1000	1456
Arithmetic 00	138	1136	1208	1664

additional handoffs reconstruct execution in a fresh process, reducing the chance that success depends solely on source-process addresses. The fixture records expected malformed-input behavior and allocation balance separately. Raw CSVs, JSON summaries, source hashes, compiler commands, and the experiment script are archived.

The phrase “expected outcomes” needs care. Incorrect context lengths were rejected without changing the destination; same-length corrupted scratch data and invalid instruction positions were accepted by the unpacker. The test did not execute the invalid instruction position. Tested malformed image headers and truncated images were rejected. Thus passing this suite includes confirming limitations; it does not mean all malformed continuations are safely rejected.

B. Representation Size

Table III distinguishes transferable images from loaded allocations. The host demo image is 1,695 bytes, making image plus context 2,223 bytes before framing. In LTTit’s eight-byte framing convention the corresponding arithmetic would be 2,231 bytes; this is not a measurement of that host image running on the board.

Peak RAM can additionally include the original image, packed context, combined RPC buffer, protocol queues, stacks, allocator overhead, and compiler workspaces. Earlier blog estimates of running small programs in roughly one or two KiB are workload-specific development observations. They do not include every concurrent allocation in a migration, and cannot establish that the whole LTTit system uses that much RAM.

C. Primitive Timing

The recorded platform is Windows 11 with GCC 13.1.0. Timing uses `QueryPerformanceCounter`, three discarded warm-up batches, and 31 measured batches per

TABLE IV
HISTORICAL WARM-CACHE HOST PRIMITIVE TIMES (NS PER OPERATION).
THESE EXCLUDE NETWORK TRANSFER AND ARE NOT MCU DOWNTIME
MEASUREMENTS.

Operation	-Os	-O2
Pack and free	19.7	17.1
Unpack	17.1	17.9
Load and unload	356.0	274.0
Complete fixture execution	556.1	474.4

workload and operation. Most batches contain 1,000 calls; load/unload uses 100. Table IV reports the host-demo medians of elapsed batch time divided by count.

Packing includes allocation, copy, observation, and free; loading includes unloading. Complete execution uses output hashing and continues locally at migration requests, excluding terminal output and deliberate demo delays. CPU affinity and frequency were not locked, and optimization-level runs were sequential. These values characterize warm-cache primitives; they are not a causal comparison of compiler strategies or individual-call latency tails. End-to-end downtime includes suspension, queueing, transport, destination scheduling, loading, and resumption, which this table does not measure.

D. What the Evidence Establishes

The corpus supports compatibility of the tested compiler/runtime path and successful reconstruction of tested continuations. The existing board demonstration supports an actual cross-board execution path. Neither establishes general memory safety, arbitrary-program correctness, sustained distributed operation, energy savings, or a speedup caused by moving computation. The present consolidation reuses these records and does not label them as newly rerun experiments.

VI. THE UNFINISHED DISTRIBUTED SCHEDULING DIRECTION

Once execution can move, the question becomes whether it should move, whether a request should be executed remotely, or whether it should remain local. The author's August notes connect this to network congestion control: a compute node also has a finite service rate, requests can queue, and feedback can regulate offered work. This is the intended next research direction, not evidence that a placement controller already exists.

A. Units and the Resource Analogy

A minimal fluid queue notation in the notes is

$$\dot{q}_i = u_i - \mu_i \quad (q_i > 0), \quad (3)$$

where q_i is queued work, u_i admitted work per second, and μ_i completed work per second; reflection is needed at zero. This can describe homogeneous requests only after their work unit is defined. Counting calls treats a cheap sensor read and a long computation as equal unless normalized deliberately.

The world-tree namespace makes different resources addressable through common operations, but it does not give

them a common physical unit. Link capacity is bytes per second, CPU service is work per second, memory is bytes, and a concurrency window counts outstanding operations. A TCP-like window w suggests a rate of roughly w/T only under suitable completion timing and saturation; variable execution times and dependencies change that relationship. These distinctions are needed before a utility or fairness statement is meaningful.

Migration consumes communication and destination resources itself. Moving an execution can reduce future remote calls while increasing transfer cost and contention. The current record has no measured model that closes these quantities into a validated optimization objective. This paper therefore does not introduce an arbitrary finite-horizon assignment optimizer and attribute it to the project.

B. AIMD Exploration and an Algebraic Gap

The August 23 post explicitly recognizes prior AIMD scheduling work [9]. Its local derivation explores additive rates α_i , multiplicative factors β_i , incoming work rate λ , and a control period T . A queue-clearing calculation gives, under its balance assumptions,

$$T = \frac{2(\lambda - \beta^T u)}{A}, \quad A = \sum_i \alpha_i. \quad (4)$$

The draft then uses $u^+ = u + \alpha T$, leading to

$$u^+ = \Phi u + b, \quad \Phi = I - \frac{2}{A} \alpha \beta^T, \quad b = \frac{2\lambda}{A} \alpha. \quad (5)$$

It states the usual conditional equilibrium and spectral-radius criteria, but does not establish their premises for this matrix.

There is a concrete obstacle: for dimension greater than one, $I - \Phi$ has rank at most one and is singular. Every nonzero vector v with $\beta^T v = 0$ satisfies $\Phi v = v$, so $\rho(\Phi) \geq 1$. The displayed recurrence therefore cannot establish a unique globally attracting allocation by inverting $I - \Phi$ or asserting $\rho(\Phi) < 1$. The post also changes whether the multiplicative decrease is included in the cycle state, which must be resolved before selecting a recurrence. This is an editorial audit of the local notes, not a refutation of the cited AIMD paper, whose model has its own definitions.

The right next step is to fix those definitions and reproduce the existing model before coupling it to ccBPF migration. Merely adding an AIMD formula to migration calls would leave important mechanisms unspecified: the measured service signal, update event, inactive-task behavior, actuator, migration cost, and what happens when the source or destination fails.

VII. RELIABILITY AND SYSTEM INTEGRATION BOUNDARIES

A. Receiving Bytes is Not Owning Execution

The current demonstration stops the source invocation and sends to a fixed destination. A robust distributed protocol must additionally decide which node owns the right to continue. If a reply is lost, retrying can duplicate execution; discarding the source state too early can lose progress. Reliable transport

alone does not provide exactly-once external effects or durable migration ownership.

A future protocol needs an execution identifier, epoch or version, program/context association, destination readiness, and a recovery rule. Those are requirements, not implemented claims in this paper. A returned RPC response may acknowledge bytes or handler completion without proving that all later execution and resource effects have completed.

B. Namespace and External State

Program-local state can be copied; external resources need their own policy. A map may be shared or private, a file position may belong to a session, and a physical actuator cannot move with the bytecode. Resource rebinding can also change observed timing and errors. The June design sketches a broader resource migration interface, but a sketch of serialize/deserialize hooks is not an implementation of distributed consistency or a theorem of transparent migration.

CCNET could eventually expose versioned connectivity and service estimates; SCP could expose delivered-byte and delay observations; CCRPC could name a destination execution resource. ccBPF would remain the continuation mechanism. Combining them still requires application-level work units and ownership semantics. A rate forecast from an ideal fair tree cannot be substituted directly for measured migration goodput, just as a local VM instruction budget cannot be substituted for CPU admission capacity.

C. A Bounded Next Evaluation

The existing substrate is sufficient to frame a small next experiment without claiming a completed scheduler: compare continuing locally, restarting remotely, and resuming remotely for the same program and inputs. Record image/context bytes, actual end-to-end completion time, peak RAM, resource effects, and retries. Only then evaluate a clearly specified policy under stable and changing service rates. That experiment has not been performed in the preserved corpus.

Safety evaluation also needs to distinguish trusted cooperative extensions from untrusted bytecode or snapshots. Context length checks alone do not validate instruction positions, memory invariants, or compatible program versions. Compiler restrictions and an interpreter reduce the implementation scope, but they do not automatically confer a formal security guarantee.

VIII. CONCLUSION AND ARTIFACT STATUS

The completed work is a concrete execution substrate: restricted source compilation, dynamic invocation, explicit resumable state, and a demonstrated path through LTTit's file-style RPC. The archived host corpus makes its representation and continuation behavior inspectable. The research has also identified a possible connection between computational load and congestion-control ideas, while leaving the scheduling model unresolved.

This distinction preserves the author's actual progress. Execution migration makes placement research possible; it does not solve placement. Future claims should be attached to a specified model, a completed proof or test, and the implementation that was actually measured. No general distributed scheduler, fault-tolerant ownership protocol, or cluster-wide performance improvement is asserted here.

The implementation and October 4 results belong to the author's original ccBPF repository and research records; copied snapshots are not bundled with this self-contained manuscript. Independent reproduction requires those original artifacts. This paper records execution migration and unfinished scheduling research without attributing an unrelated placement prototype to the project.

REFERENCES

- [1] S. McCanne and V. Jacobson, "The BSD Packet Filter: A new architecture for user-level packet capture," in *Winter USENIX*, 1993, pp. 259–269.
- [2] P. Levis and D. Culler, "Maté: A tiny virtual machine for sensor networks," in *ASPLOS*, 2002, pp. 85–95. doi:10.1145/605397.605407.
- [3] C.-L. Fok, G.-C. Roman, and C. Lu, "Agilla: A mobile agent middleware for self-adaptive wireless sensor networks," *ACM TAAS*, vol. 4, no. 3, Art. 16, 2009. doi:10.1145/1552297.1552299.
- [4] A. Fuggetta, G. P. Picco, and G. Vigna, "Understanding code mobility," *IEEE Trans. Software Engineering*, vol. 24, no. 5, pp. 342–361, 1998. doi:10.1109/32.685258.
- [5] K. Zandberg and E. Baccelli, "Minimal virtual machines on IoT micro-controllers: The case of Berkeley Packet Filters with rBPF," in *IEEE PEMWN*, 2020. Author version: arXiv:2011.12047.
- [6] K. Zandberg et al., "Femto-containers: Lightweight virtualization and fault isolation for small software functions on low-power IoT microcontrollers," in *Middleware*, 2022, pp. 161–173. doi:10.1145/3528535.3565242.
- [7] R. Gurdeep Singh and C. Scholliers, "WARDuino: A dynamic WebAssembly virtual machine for programming microcontrollers," in *MPLR*, 2019, pp. 27–36. doi:10.1145/3357390.3361029.
- [8] S. Yuan et al., "End-to-end mechanized proof of an eBPF virtual machine for micro-controllers," in *CAV, LNCS 13372*, 2022, pp. 293–316. doi:10.1007/978-3-031-13188-2_15.
- [9] E. Vlahakis, N. Athanasopoulos, and S. McLoone, "AIMD scheduling and resource allocation in distributed computing systems," 2021. Author manuscript: <https://arxiv.org/abs/2109.02589>.